RAG Pipeline - Strategy: Choosing an Advanced Indexing Method

1. Overview: Sibling Strategies

Multi-Representation, **RAPTOR**, and **ColBERT** are not sequential steps; they are **alternative strategies** (siblings). You typically choose one based on the nature of your documents and the questions users will ask.

- **Multi-Representation:** Decouples the *embedded summary* from the *retrieved content*.
- **RAPTOR:** Builds a *tree* of summaries to answer high-level questions.
- **ColBERT:** Keeps *every token* as a vector to answer extremely precise, low-level questions.

2. Option A: Multi-Representation (The "Context" Choice)

Use this when your documents are distinct entities (e.g., contracts, separate articles) and the goal is to find the *one* right document, even if it's dense.

- **Best For:** "Needle in a haystack" retrieval where the needle is a *concept*, not just a keyword.
- **Goal:** "I want to find the specific document that answers this question, regardless of how long or complex that document is."
- **Mechanism (1-to-1 Mapping):**
  - Creates a summary (or child chunks) for *one* document.
  - Links that summary back to that *one* document.
- **Example Use Case:** A library of 1,000 distinct employee contracts.
  - **Query:** "Does John Smith's contract include dental?"
  - **Result:** The system matches the summary of "John Smith's Contract" and retrieves the full text.

3. Option B: RAPTOR (The "Synthesis" Choice)

Use this when your documents are highly interconnected, and users need "big picture" answers that span *across* them.

- **Best For:** Thematic understanding and summarization.
- **Goal:** "I want to understand trends, themes, or commonalities across my entire dataset."
- **Mechanism (Many-to-1 Mapping):**
  - Clusters parts of *many* different documents together based on topic.
  - Creates a summary for that *cluster*.
  - Builds a hierarchy (Tree) to connect details to high-level themes.
- **Example Use Case:** A collection of 500 medical trial reports.

- **Query:** "What are the common side effects reported for Drug X across all trials?"
- **Result:** The system matches a "Side Effects" summary node which was created by synthesizing findings from 50 different reports.

4. Option C: ColBERT (The "Precision" Choice)

Use this when your data involves complex terminology, code, or specific names where **every single word matters**, and standard "dense" retrieval is too fuzzy.

- **Best For:** Fine-grained, token-level matching.
- **Goal:** "I need to distinguish between 'The dog bit the man' and 'The man bit the dog', or find a specific error code in a log."
- **Mechanism (Late Interaction):**
  - Stores a vector for *every token* (word) in the document.
  - Compares every token in the query against every token in the doc (MaxSim).
- **Example Use Case:** Technical Documentation or Legal Search.
  - **Query:** "Error 404 in module `auth.py`"
  - **Result:** ColBERT finds the exact line because it matches the tokens `404`, `module`, and `auth.py` directly, whereas a standard retriever might just return a generic page about "errors."

5. Decision Matrix

Feature	Multi-Representation	RAPTOR	ColBERT
Complexity	Moderate	High (Clustering/Trees)	Moderate (Standard Retriever API)
Indexing Cost	Medium (1 LLM call per doc)	High (Many LLM calls)	High (Large Index Size on Disk)
Query Type	Factual, document-centric.	Abstract, thematic, corpus-centric.	Precise, terminology-heavy, technical.
Data Structure	Distinct, independent docs.	Interconnected knowledge base.	Complex jargon, code, or nuances.
"When to use?"	**Start here.** Good default.	Use for **Synthesis** & **High-Level** Qs.	Use for **Precision** & **Fine Details**.